

Конфликты переменных в JavaScript

Пусть у нас есть HTML страница index.html, на которой в теге script вы создаете переменную str и выводите ее на экран:

Index.html

```
1 <html>
2   <head>
3     <script>
4       let str = 'script text';
5       alert(str); // выведет 'script text'
6     </script>
7   </head>
8   <body>
9     ...
10  </body>
11 </html>
```

Пусть у нас также есть файл script.js, в котором также задается переменная str:

script.js

```
1 let str = 'file text';
```

Пусть теперь наш файл script.js подключается к странице index.html следующим образом:

```
1 <html>
2   <head>
3     <script>
4       let str = 'script text';
5     </script>
6     <script src="script.js"></script>
7     <script>
8       alert(str);
9     </script>
10  </head>
11  <body>
12    ...
13  </body>
14 </html>
```

Так как переменная str есть и в файле index.html, и в файле script.js, то будет конфликт, в котором победит та переменная, которая написана ниже, то есть переменная из файла script.js. То есть наш код выведет 'file text', а не 'script text', как мы ожидаем.

Проблема на самом деле очень серьезная. В реальном сайте у вас чаще всего будет несколько файлов с вашими скриптами, кроме того, вы будете подключать какие-то сторонние плагины. В этом случае переменные и функции одного файла могут конфликтовать с переменными и функциями другого файла.

Модули через замыкания в JavaScript

Описанная выше проблема характерна для любого языка программирования. В качестве решения применяют так называемые *модули*.

Модуль представляет собой некую конструкцию, сделанную так, чтобы переменные и функции этой конструкции были видны только внутри нее и не мешали никому снаружи.

В JavaScript существуют несколько типов модулей. Самые простые *модули через замыкания* создаются с помощью вызова функции на месте, вот так:

```
1 ;(function() {  
2     // тут код модуля  
3 })();
```

Переменные и функции, созданные в таком модуле, не будут видны снаружи этого модуля:

```
1 ;(function() {  
2     let str = 'переменная модуля';  
3  
4     function func() {  
5         alert('функция модуля');  
6     }  
7 })();  
8  
9 // Тут переменные и функции модуля недоступны:  
10 alert(str);  
11 alert(func);
```

Практическое применение

Пусть у нас даны два дива с числами:

```
1 <div id="div1">10</div>  
2 <div id="div2">10</div>
```

Давайте сделаем так, чтобы по клику по первому диву его значение возводилось в квадрат, а по клику по второму диву - в куб.

Организуем наш код в виде двух модулей:

```
1 ;(function() {  
2     let elem = document.querySelector('#div1'); // первый див  
3  
4     function func(num) {  
5         return num * num; // возведем в квадрат  
6     }  
7  
8     elem.addEventListener('click', function() {  
9         this.textContent = func(elem.textContent);  
10    });  
11 })();  
12
```



```

12
13 ;(function() {
14     let elem = document.querySelector('#div2'); // второй див
15
16     function func(num) {
17         return num * num * num; // возведем в куб
18     }
19
20     elem.addEventListener('click', function() {
21         this.textContent = func(elem.textContent);
22     });
23 })();

```

Теперь в каждом из модулей мы можем использовать любые переменные и функции, не боясь того, что они будут конфликтовать с другими переменными и функциями нашего скрипта.

Например, оба элемента мы храним в переменной `elem` - каждый в своей переменной своего модуля. Если бы модулей здесь не было, пришлось бы вводить разные переменные для хранения наших элементов. А с модулями мы можем спокойно использовать нашу переменную, не боясь того, что кто-то захочет также использовать эту переменную.

Передача параметров в модуль через замыкания в JavaScript

Хорошей практикой считается не зашивать какие-то значения в модуль, а передавать их параметром самого модуля (то есть параметром вызывающейся на месте функции):

```

1 ;(function(arg1, arg2) { // параметры попадают в переменные
2
3 })(1, 2); // передаем какие-то параметры

```

Давайте посмотрим на примере. Пусть у нас есть див с числом и кнопка:

```

1 <div id="div">3</div>
2 <button id="btn">click me</button>

```

Пусть у нас также есть некоторый модуль:

```

1 ;(function() {
2     let div = document.querySelector('#div');
3     let btn = document.querySelector('#btn');
4
5     function func(num) {
6         return num * num;
7     }
8
9     btn.addEventListener('click', function() {
10         div.textContent = func(div.textContent);
11     });
12 })();

```

Как вы видите, селекторы наших элементов жестко зашиты в коде модуля. Более удачным решением будет передать их параметрами модуля - так в дальнейшем мы легко сможем их изменить. Исправим наш модуль:

```

1 |;(function(selector1, selector2) {
2 |    let div = document.querySelector(selector1);
3 |    let btn = document.querySelector(selector2);
4 |
5 |    function func(num) {
6 |        return num * num;
7 |    }
8 |
9 |    btn.addEventListener('click', function() {
10 |        div.textContent = func(div.textContent);
11 |    });
12 |})('#div', '#btn');

```

№ 1

©jsPmMCP

Дана кнопка и три инпута, в которые вводятся числа. По нажатию на кнопку выведите в консоль сумму введенных чисел. Реализуйте задачу с помощью модуля.

Передача родительского элемента в модуль через замыкания в JavaScript

Пусть у нас есть следующие элементы:

```

1 |<div id="div1">1</div>
2 |<div id="div2">2</div>
3 |<div id="div3">3</div>
4 |<div id="res"></div>
5 |
6 |<button id="btn">click me</button>

```

Пусть с этими элементами работает следующий модуль:

```

1 |;(function(selector1, selector2, selector3,
2 |    selector4, selector5) {
3 |    let div1 = document.querySelector(selector1);
4 |    let div2 = document.querySelector(selector2);
5 |    let div3 = document.querySelector(selector3);
6 |    let res = document.querySelector(selector4);
7 |    let btn = document.querySelector(selector5);
8 |
9 |    btn.addEventListener('click', function() {
10 |        let num1 = Number(div1.textContent);
11 |        let num2 = Number(div2.textContent);
12 |        let num3 = Number(div3.textContent);
13 |
14 |        res.textContent = num1 + num2 + num3;
15 |    });
16 |})('#div1', '#div2', '#div3', '#res', '#btn');

```


Как вы видите, количество передаваемых в модуль параметров несколько великовато и создает неудобства. Обычно в этом случае поступают следующим образом: передают в модуль общего родителя всех элементов, с которыми работает наш модуль, а уже внутри модуля получают из этого родителя различные элементы.

Давайте сделаем нашим тегам общего родителя:

```
1 <div id="parent">
2   <div id="div1">1</div>
3   <div id="div2">2</div>
4   <div id="div3">3</div>
5   <div id="res"></div>
6
7   <button id="btn">click me</button>
8 </div>
```

Переделаем теперь код модуля:

```
1 ;(function(root) {
2   let parent = document.querySelector(root);
3
4   let div1 = parent.querySelector('#div1');
5   let div2 = parent.querySelector('#div2');
6   let div3 = parent.querySelector('#div3');
7
8   let res = parent.querySelector('#res');
9   let btn = parent.querySelector('#btn');
10
11   btn.addEventListener('click', function() {
12     let num1 = Number(div1.textContent);
13     let num2 = Number(div2.textContent);
14     let num3 = Number(div3.textContent);
15
16     res.textContent = num1 + num2 + num3;
17   });
18 })( '#parent');
```

Таким образом получится, что родительский элемент мы передаем в модуль снаружи и легко можем его поменять. А дочерние элементы являются внутренним делом модуля.

Передача настроек модуля через замыкания в JavaScript

Пусть у нас есть следующий модуль:

```
1 ;(function(root, type, amount) {
2   let parent = document.querySelector(root);
3
4   for (let i = 1; i <= amount; i++) {
5     let elem = document.createElement(type);
6     parent.append(elem);
7   }
8 })( '#parent', 'p', 5);
```


Как вы видите, в этот модуль передаются три настройки: селектор родительского элемента, тип элемента для создания и количество элементов.

Как правило такие настройки делают в виде объекта:

```
1 | let config = {  
2 |   root: '#parent',  
3 |   type: 'p',  
4 |   amount: 5  
5 | }
```

Давайте передадим параметром модуля наш объект:

```
1 | ;(function(config) {  
2 |   let parent = document.querySelector(config.root);  
3 |  
4 |   for (let i = 1; i <= config.amount; i++) {  
5 |     let elem = document.createElement(config.type);  
6 |     parent.append(elem);  
7 |   }  
8 | })(config);
```

Более принято выполнять деструктуризацию объекта с настройками:

```
1 | ;(function({root, type, amount}) {  
2 |   let parent = document.querySelector(root);  
3 |  
4 |   for (let i = 1; i <= amount; i++) {  
5 |     let elem = document.createElement(type);  
6 |     parent.append(elem);  
7 |   }  
8 | })(config);
```

Параметры по умолчанию

Пусть мы хотим разрешить при использовании модуля не указывать все настройки. Если какая-то из настроек не будет указана, то она примет значение по умолчанию.

К примеру, в нашем случае можно сделать так, чтобы тип по умолчанию принимал значение p, а количество - значение 5:

```
1 ;(function({root, type = 'p', amount = 5}) {  
2   let parent = document.querySelector(root);  
3  
4   for (let i = 1; i <= amount; i++) {  
5     let elem = document.createElement(type);  
6     parent.append(elem);  
7   }  
8 })(config);
```

В этом случае мы легко можем по-разному конфигурировать наш модуль. К примеру, укажем только родительский элемент:

```
1 let config = {  
2   root: '#parent',  
3 }
```

А теперь укажем родительский элемент и количество. При этом нам не нужно будет указывать тип - ведь элементы объекта настроек не имеют порядка, и мы можем опускать их как угодно. Итак, вот наша настройка:

```
1 let config = {  
2   root: '#parent',  
3   amount: 10  
4 }
```

Экспорт переменных и функций в модулях через замыкания в JavaScript

Иногда нужно сделать так, чтобы некоторые переменные и функции модуля были доступны снаружи. Давайте посмотрим, как это делается. Пусть у нас есть следующий модуль:

```
1 ;(function() {  
2   let str = 'переменная модуля';  
3  
4   function func() {  
5     alert('функция модуля');  
6   }  
7 })();
```

Давайте экспортируем нашу функцию func. Для этого запишем ее в свойство встроенного в браузер объекта window:

```
1 ;(function() {  
2   let str = 'переменная модуля';  
3  
4   function func() {  
5     alert('функция модуля');  
6   }  
7  
8   window.func = func;  
9 })();
```

Теперь мы можем вызвать нашу функцию снаружи модуля:

```
1 ;(function() {  
2   let str = 'переменная модуля';  
3  
4   function func() {  
5     alert('функция модуля');  
6   }  
7  
8   window.func = func;  
9 })();  
10  
11 window.func(); // выведет 'функция модуля'
```

При этом не обязательно вызывать функцию как свойство объекта window:

```
1 ;(function() {  
2   let str = 'переменная модуля';  
3  
4   function func() {  
5     alert('функция модуля');  
6   }  
7  
8   window.func = func;  
9 })();  
10  
11 func(); // выведет 'функция модуля'
```

Дан следующий модуль:

```
1  ;(function() {  
2    let str1 = 'переменная модуля';  
3    let str2 = 'переменная модуля';  
4    let str3 = 'переменная модуля';  
5  
6    function func1() {  
7      alert('функция модуля');  
8    }  
9    function func2() {  
10     alert('функция модуля');  
11   }  
12   function func3() {  
13     alert('функция модуля');  
14   }  
15 })();
```

Экспортируйте наружу одну из переменных и две любые функции.

Экспорт объекта в модулях через замыкания в JavaScript

Пусть у нас есть следующий модуль:

```
1  ;(function() {  
2    function func1() {  
3      alert('module function');  
4    }  
5    function func2() {  
6      alert('module function');  
7    }  
8    function func3() {  
9      alert('module function');  
10   }  
11 })();
```

Пусть мы хотим экспортировать наружу все три функции. В этом случае, чтобы не плодить снаружи модуля лишних имен функций, лучше записать все функции в один объект и выполнить экспорт этого объекта:

```
1  ;(function() {  
2    function func1() {  
3      alert('module funcion');  
4    }  
5    function func2() {  
6      alert('module funcion');  
7    }  
8    function func3() {  
9      alert('module funcion');  
10   }  
11  
12   window.module = {func1: func1, func2: func2, func3: func3};  
13 })();
```

Так как имена ключей и переменных совпадают, то объект с функциями можно упростить:

Можно пойти и другим путем. Будем записывать функции в объект сразу при описании функции, вот так:

```
1  ;(function() {  
2    let module = {};  
3  
4    module.func1 = function() {  
5      alert('module function');  
6    }  
7    module.func2 = function() {  
8      alert('module function');  
9    }  
10   module.func3 = function() {  
11     alert('module function');  
12   }  
13  
14   window.module = module;  
15 })();
```

№ 1

©jsPmMCVFEO

Дан следующий модуль:

```
1  ;(function() {  
2    let str1 = 'module variable';  
3    let str2 = 'module variable';  
4    let str3 = 'module variable';  
5  
6    function func1() {  
7      alert('module function');  
8    }  
9    function func2() {  
10     alert('module function');  
11   }  
12   function func3() {  
13     alert('module function');  
14   }  
15   function func4() {  
16     alert('module function');  
17   }  
18   function func5() {  
19     alert('module function');  
20   }  
21 })();
```

Экспортируйте наружу объект с первыми пятью функциями и первыми двумя переменными

Библиотеки через замыкания в JavaScript

Часто в JavaScript создаются *библиотеки*, представляющие собой наборы функций для пользования другими программистами.

Такие библиотеки обычно оборачиваются в модули через замыкания. Это делается для того, чтобы при подключении библиотеки во внешнем мире появлялось как можно меньше функций.

Как правило каждая библиотека старается создавать во внешнем мире только одну переменную - объект с функциями библиотеки. При этом внутри в коде библиотеки часть функций являются основными, а часть - вспомогательными. Очевидно, что во внешний мир мы хотим экспортировать только нужные функции, не захламляя экспортируемый объект вспомогательными функциями.

Давайте посмотрим на примере. Пусть у нас есть следующий набор функций, которые мы хотели бы превратить в библиотеку:

```
math.js
1 function square(num) {
2   return num ** 2;
3 }
4 function cube(num) {
5   return num ** 3;
6 }
7 function avg(arr) {
8   return sum(arr, 1) / arr.length;
9 }
10 function digitsSum(num) {
11   return sum(String(num).split(''));
12 }
13
14 // вспомогательная функция
15 function sum(arr) {
16   let res = 0;
17
18   for (let elem of arr) {
19     res += +elem;
20   }
21
22   return res;
23 }
```

Давайте оформим наши функции в виде модуля:

math.js

```
1  ;(function() {  
2    function square(num) {  
3      return num ** 2;  
4    }  
5    function cube(num) {  
6      return num ** 3;  
7    }  
8    function avg(arr) {  
9      return sum(arr, 1) / arr.length;  
10   }  
11   function digitsSum(num) {  
12     return sum(String(num).split(''));  
13   }  
14 }
```

```

14
15 // вспомогательная функция
16 function sum(arr) {
17     let res = 0;
18
19     for (let elem of arr) {
20         res += +elem;
21     }
22
23     return res;
24 }
25 })();

```

А теперь экспортируем все функции, кроме вспомогательной:

А теперь экспортируем все функции, кроме вспомогательной:

math.js

```

1 ;(function() {
2     function square(num) {
3         return num ** 2;
4     }
5     function cube(num) {
6         return num ** 3;
7     }
8     function avg(arr) {
9         return sum(arr, 1) / arr.length;
10    }
11    function digitsSum(num) {
12        return sum(String(num).split(''));
13    }
14
15    // вспомогательная функция
16    function sum(arr) {
17        let res = 0;
18
19        for (let elem of arr) {
20            res += +elem;
21        }
22
23        return res;
24    }
25
26    window.math = {square, cube, avg, digitsSum};
27 })();

```

Пусть у нас есть HTML страница index.html:

Index.html

```
1 <html>
2   <head>
3     <script>
4
5     </script>
6   </head>
7 </html>
```

Подключим к ней нашу библиотеку:

Index.html

```
1 <html>
2   <head>
3     <script src="math.js"></script>
4     <script>
5
6     </script>
7   </head>
8 </html>
```

Воспользуемся функциями из нашей библиотеки:

Index.html

```
1 <html>
2   <head>
3     <script src="math.js"></script>
4     <script>
5       alert(math.avg([1, 2, 3]) + math.square());
6     </script>
7   </head>
8 </html>
```

№1

© jsPmMCLb

Дан следующий код:

```
1  function avg1(arr) {  
2    return sum(arr, 1) / arr.length;  
3  }  
4  
5  function avg2(arr) {  
6    return sum(arr, 2) / arr.length;  
7  }  
8  
9  function avg3(arr) {  
10   return sum(arr, 3) / arr.length;  
11 }  
12  
13 // вспомогательная функция  
14 function sum(arr, pow) {  
15   let res = 0;  
16  
17   for (let elem of arr) {  
18     res += elem ** pow;  
19   }  
20  
21   return res;  
22 }
```

Оформите этот код в виде модуля. Ээкспортируйте наружу все функции, кроме вспомогательной.

№2

© jsPmMCLb

Изучите библиотеку [underscore](#). Сделайте свою аналогичную библиотеку, повторив в ней 5-10 функций оригинальной библиотеки.